

Data Engineer - Case Study

Context

The data team at Octopus are responsible for building robust, reliable, well documented and scalable pipelines to move data from source to target and get it in shape to be analysed. This often requires combining data from multiple data sources as well as cleaning it, to get a reliable [single source of truth](#).

Tasks

Our pricing data analysts want to be able to calculate how our products would rank, based on their prices, on price comparison websites (PCS), as they are the main source of customers. These websites receive data from all electricity providers in the market and aggregate them, showing the user all electricity tariffs in the market, ranked from lowest to highest price.

The analysts are interested in tracking this especially for our main product, Octopus 12 (with `product_id = 224`). To achieve this we would like you to:

1. Build a data pipeline that takes price comparison website data, network costs data and the static DB data and moves it into a single datastore that the data analysts can query using SQL
2. Build a final analytics table in the same datastore for the analysts to use. The table should have one row per `report_date`, `postcode`, `location_name`, `annual_consumption_in_kWh` and `market` and should show the corresponding ranking of the product, plus its gross price.

For a given annual consumption (kWh), the monthly gross price should be based on:

- The network costs from NetworkC,
 - Net variable costs (per kWh)
 - Net fixed costs (per year)
- Our product rates,
 - Net variable rate (per kWh)
 - Net standing charge (per month)
- And the underlying VAT of 19%.

$$gross_price_{month} = [(1 + tax_rate) * (network_fixed_cost_{year} + network_variable_cost * consumption_{year} + product_standing_charge_{year} + product_variable_rate * consumption)] / 12$$

Data

We've provided you with four data sources:

- CSV files from the price comparison website, Compare24. These files are daily exports of prices as shown on their website, with naming `price_comparison_{date}.csv`. They contain prices (`gross_price_per_month`) at the level of:
 - `postcode`
 - `location` - for this exercise, it's been restricted to Berlin and Munich
 - `annual_consumption_in_kWh`: this is the consumption the `gross_price_per_month` has been calculated on. You can assume that annual consumption is divided equally across all months.
 - `market`: *bonus* or *non-bonus*. In the bonus markets, companies offer bonuses (one time sign-up bonus, or similar) to customers as a way to improve their rankings. Non-bonus markets are purely based on the usual tariff prices.
 - `ranking`: Compare24 takes all tariffs from all providers and ranks them according to their price. This is linear - the higher the price, the lower the ranking. The higher a company ranks, the closer to the top in the webpage they appear.
- CSV file from our market research tool, NetworkC (`network_costs.csv`). This data shows how much Octopus needs to pay for the use of the electricity network, to distribute electricity to each customer that signs up. These costs are divided into fixed yearly costs that don't depend on consumption, and a unit cost per kWh. This table is at the level of:
 - `postcode`

- `town`
- `network_operator`
- A sqlite database `backend.db` containing tables from our system's backend:
 - `products` - these are the tariffs that can be sold to customers, with an associated code and description. The code shows whether it's an electricity or gas tariff, the name of the product itself and whether it's a 12-month or 24-month product. All products we picked are fixed (the price doesn't depend on the time of the day).
 - `product_rates` - this table has intervals (defined by *valid_from*, *valid_to*) of each product, given by *product_id*, that show the price per unit (kWh, kilowatt-hour) of each tariff.
- A sql lite database `dwh.db` containing the result of Task 1, only needed in case you are stuck and want to proceed with Task 2

Hints

- You should assume that the CSV files from the price comparison website come in every day before 8am
- You should assume the database gets updated once a day at midnight

Guidance

- You should spend around 4 hours on the problem. If you can't finish, don't worry, just write down what you would do
- This isn't a test of electricity industry knowledge so if anything is unclear just ask
- Your pipeline should be written in Python but feel free to use frameworks, libraries or other tools
- That said, this isn't a test of which libraries you know. Credit will be given for simple, reliable and well engineered solutions.
- Make the code as production ready as possible (within reasonable limits given the time constraints)
- Feel free to choose your own database or storage format for the target
- Your pipeline should be built in a way that someone else could deploy and maintain it easily
- Although the data volumes for this task are small you should think about how your pipeline might scale as data volumes grow
- You should be able to automate your pipeline or at least be able to describe how you would automate it
- Do not share your solution on Github/Gitlab, as it would give an advantage to other candidates
- If you are stuck with Task 1 and want to continue with Task 2, you can use the intermediate result in `dwh.db`